Arquitectura de Monolegal Billing

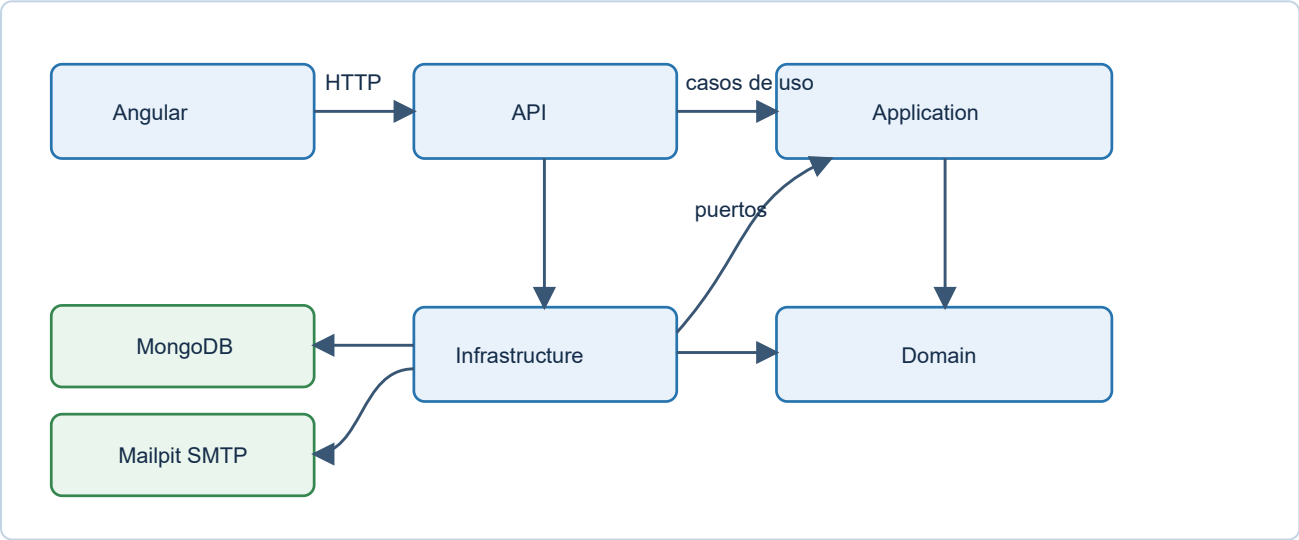
Demostración local de recordatorios por factura · .NET 10, Angular 22, MongoDB y Mailpit

Objetivo y alcance

La aplicación consulta facturas de tres clientes ficticios y permite iniciar manualmente desde Angular un lote de recordatorios. Cada factura elegible genera un mensaje individual capturado por Mailpit. El estado de esa factura solo se actualiza después de que el servidor SMTP acepta el mensaje y MongoDB confirma una escritura condicional.

> Es una demostración local: la aceptación por Mailpit no acredita entrega al destinatario. No publicar el endpoint de procesamiento sin autenticación y autorización.

Componentes y dependencias



Domain contiene entidades y política de transición sin dependencias externas. Application coordina el lote mediante puertos pequeños. Infrastructure implementa persistencia MongoDB y envío con MailKit; API compone dependencias, expone contratos HTTP, ProblemDetails y OpenAPI. Angular consume únicamente la API.

Decisiones técnicas y modelo de datos

Decisión	Justificación y consecuencia
Monolito modular	Separa responsabilidades sin introducir microservicios ni infraestructura ajena al alcance.
MongoDB con seed manual	Colecciones e índices explícitos; tres clientes y seis facturas reproducibles sin sobrescribir estados procesados al repetir el seed.
Mailpit local	Prueba SMTP con destinatarios ficticios `example.test`, sin cuentas ni credenciales reales.
Sin scheduler	Un botón Angular confirma el alcance global y llama al POST; no se ejecutan lotes al iniciar.

Colección	Datos relevantes
`clientes`	Identidad, nombre, correo electrónico ficticio y estado activo de muestra; índice por email.
`facturas`	Cliente, número único por cliente, importe `decimal/Decimal128`, moneda COP, fechas UTC, estado exacto y reserva opcional `processing` (token, fase y vencimiento).
`intentosNotificacion`	Identificadores de lote y factura, estados previo y previsto, fase del intento y resultados; índices para consultar por factura y lote.

Seed inicial: dos facturas en `primerrecordatorio`, dos en `segundorecordatorio`, una `pendiente` y una `desactivado`. Solo los dos primeros estados son elegibles; cada lote avanza como máximo una vez por factura.

Orden del procesamiento y límite de consistencia

1. Obtener instantánea de identificadores elegibles; reservar cada factura con una operación atómica y un token.
2. Validar cliente y correo, calcular el siguiente estado sin modificar aún la factura y registrar el intento en fase `Dispatching` antes de llamar a SMTP.
3. Enviar exactamente un mensaje por intento. Un rechazo SMTP conocido deja el estado intacto; una respuesta incierta se marca para revisión, sin reenvío automático.
4. Tras aceptación SMTP, actualizar el estado por coincidencia de ID, estado anterior y token; marcar `Completed` solo cuando la transición queda confirmada.

Un claim vencido que alcanzó `Dispatching` no se reutiliza automáticamente. MongoDB y SMTP no forman una transacción distribuida: ante confirmación ambigua o error posterior al envío se consulta Mailpit y la auditoría para reconciliar manualmente, sin declarar éxito ni prometer entrega exactamente una vez.

API, seguridad y pruebas

`GET /api/customers`, `GET /api/invoices` (filtros y paginación), `GET /api/invoices/summary`, `GET /api/reminders/runs/{batchId}` y `GET /health` permiten la consulta. `POST /api/reminders/process` inicia el lote global solo en Development. Swagger UI está disponible en `http://localhost:5091/swagger` únicamente en Development.

La API no carga archivos `.env`; configura opciones mediante `appsettings.Development.json` y variables del proceso. MongoDB y Mailpit se exponen únicamente en loopback. CORS permite solo `http://localhost:4200` en Development. El adaptador SMTP restringe host local, ausencia de TLS/credenciales y correos ficticios. Antes de un despliegue público son imprescindibles autenticación, autorización y controles adicionales.

Existen pruebas xUnit de reglas, coordinación, seed, concurrencia y contratos HTTP; pruebas Angular de servicios, confirmación, estados y tabla. Las pruebas de integración usan MongoDB real cuando Docker está disponible. La CI previa al cambio de mensajes/UX pasó backend, frontend y E2E de API en GitHub Actions (commit 141c130); esa ejecución no verifica los cambios locales actuales ni automatiza clics de navegador.

Arranque local y comprobación

1. Instalar .NET SDK 10, Node.js 24, npm y Docker Desktop con Compose.
2. Desde la raíz: `docker compose up -d`; `dotnet build Monolegal.Billing.slnx`; `dotnet run --project src/Monolegal.Billing.Api -- --seed`.
3. Iniciar la API con `dotnet run --project src/Monolegal.Billing.Api --launch-profile http` o F5 en Visual Studio con perfil `http`.
4. En `frontend/monolegal-billing`: `npm ci` y `npm start -- --host localhost --port 4200`.
5. Comprobar Angular en `http://localhost:4200` y Mailpit en `http://localhost:8025`; primer lote: cuatro correos, segundo: dos, tercero: cero. Ejecutar `dotnet test Monolegal.Billing.slnx -c Release` y `npm test -- --watch=false` en sus directorios.

Para restaurar únicamente los datos del seed en Development: `dotnet run --project src/Monolegal.Billing.Api -- --seed --reset`. Para vaciar mensajes de prueba: `DELETE http://localhost:8025/api/v1/messages`. El README ofrece la secuencia completa y sus advertencias.

Limitaciones conocidas

Sin autenticación productiva; no existe reconciliación automática entre SMTP y MongoDB; un lote sin candidatas devuelve un `batchId` cuyo historial responde 404 porque no se guardan ejecuciones vacías. Los filtros del dashboard no restringen el procesamiento global.